

Algorithm for file updates in Python

Project description

In this project, I developed a Python algorithm to automate updates to an allow list of IP addresses used by a healthcare company. The allow list identifies employees who are authorized to access restricted patient record systems. The algorithm compares the allow list against a remove list containing IP addresses that should no longer have access. After identifying matching IP addresses, the program removes them and updates the original file with the revised list. This project demonstrates my understanding of file handling, loops, conditional statements, and list manipulation in Python.

Open the file that contains the allow list

```
: import_file = "allow_list.txt"

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

with open(import_file, "r") as file:
```

This code begins by assigning the file name "allow_list.txt" to the variable `import_file`. The `open()` function is used to open the file, and the "r" argument specifies that the file should be opened in read mode. The `with` statement is used to safely handle the file because it automatically closes the file after the block of code is completed. The variable `file` temporarily stores the opened file object while it is being used inside the `with` statement.

Read the file contents

```
import_file = "allow_list.txt"

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

with open(import_file, "r") as file:

    ip_addresses = file.read()

print(ip_addresses)
```

The `.read()` method is used to read the entire contents of the file and convert them into a string. The string is then stored in the variable `ip_addresses`. This allows the program to work with the file contents inside the Python script.

Convert the string into a list

```
import_file = "allow_list.txt"

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

with open(import_file, "r") as file:

    ip_addresses = file.read()

ip_addresses = ip_addresses.split()

print(ip_addresses)
```

The `.split()` method is used to convert the string of IP addresses into a list. By default, `.split()` separates the string wherever there is whitespace, such as spaces or line breaks. Converting the data into a list makes it possible to remove individual IP addresses later in the algorithm.

Iterate through the remove list

```
import_file = "allow_list.txt"

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

with open(import_file, "r") as file:

    ip_addresses = file.read()

ip_addresses = ip_addresses.split()

for element in ip_addresses:

    print(element)
```

This `for` loop iterates through each item in the `remove_list`. The loop variable `element` represents the current IP address during each iteration. Using a loop allows the algorithm to compare every IP address in the remove list against the allow list.

Remove IP addresses that are on the remove list

```
import_file = "allow_list.txt"

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

with open(import_file, "r") as file:
    ip_addresses = file.read()

ip_addresses = ip_addresses.split()

for element in ip_addresses:
    if element in remove_list:
        ip_addresses.remove(element)

print(ip_addresses)
```

The conditional statement checks whether the current IP address stored in `element` exists in the `ip_addresses` list. The `in` keyword is used to test membership in the list. If the IP address exists, the `.remove()` method removes it from the allow list.

Applying the `.remove()` method in this way is possible because there are no duplicate IP addresses in the `ip_addresses` list.

Update the file with the revised list of IP addresses

```
import_file = "allow_list.txt"

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

with open(import_file, "r") as file:
    ip_addresses = file.read()

ip_addresses = ip_addresses.split()

for element in ip_addresses:
    if element in remove_list:

        ip_addresses.remove(element)

ip_addresses = " ".join(ip_addresses)

with open(import_file, "w") as file:
    file.write(ip_addresses)
```

The `.join()` method is used to convert the `ip_addresses` list back into a string. The string `"\n"` places each IP address on a new line in the file.

The file is then reopened using another `with` statement. This time, the `"w"` argument is used to open the file in write mode. The `.write()` method replaces the old file contents with the updated list of IP addresses.

Summary

This Python algorithm automates the process of updating an allow list of authorized IP addresses. First, the program opens and reads the contents of the allow list file. Next, it converts the file contents into a list so individual IP addresses can be modified. The algorithm then uses a `for` loop and a conditional statement to identify and remove unauthorized IP addresses listed in the remove list. Finally, the revised list is converted back into a string and written back to the file. This project demonstrates practical skills in Python programming, including file handling, loops, conditionals, and list operations.